

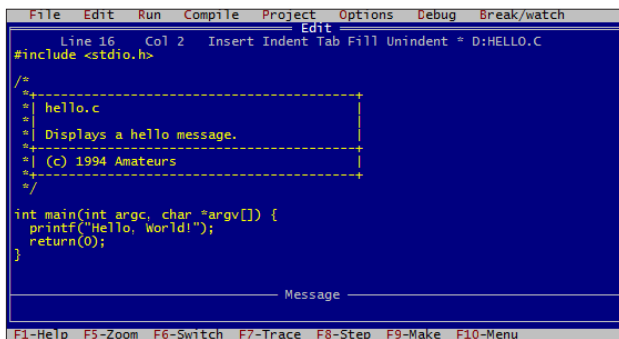


Figure 2: The spartan way of highlighting comments in source code

HTML entity references

When I started learning HTML, I found that the copyright symbol could be written as `©`. The registered symbol (®) could be written as `®` and Trademark symbol (™) could be written as `™`. HTML has several such character entity references. However, any character can be written using its Unicode codepoint, which is known as a numeric entity reference. For the copyright symbol, this is `©`. It can also be written as `©` with the *x* signifying that the codepoint is hexadecimal. Similarly, the numeric entity reference for the registered symbol is written as `®` or as `®`.

Source code file encoding

If your OS is Windows, then it is likely that you save all your source code files in the default Windows encoding *Latin-1* or *Western European*. These do not work well with non-English Unicode strings. (Ideally, you should be storing all UI strings in a resource file so that file encoding is never a problem.) However, for maximum resilience and portability, save all your source files in UTF-8 encoding. If your file turns into gibberish, you need to first select everything in the file to the system clipboard (*Ctrl+A* and *Ctrl+C*), then change the encoding, and finally do a paste (*Ctrl+V*). If your editor allows you to change the encoding in the ‘Save as’ dialog box, then paste the text after saving the file. With UTF-8 encoding, you will eliminate a whole heap of trouble.

HTML forms

If you have a Web script that outputs HTML to a browser, then let the browser know the encoding of the output stream. In PHP, the first line could be:

```
<?php header('Content-type: text/html; charset=utf-8'); ?>
```

This is not enough. In the HTML, declare the encoding as early as possible:

```
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html;
    charset=UTF-8" />
    <title>This page is Unicode-encoded</title>
  ...
```

This is particularly important if you have HTML forms where you accept text data. If your HTML page is not Unicode-encoded or if the server-side script is not saved with Unicode encoding, then you may not always be able to correctly process non-English text input.

Fonts

Despite the above, if you still get the dreaded question marks or boxy characters, it means you do not have the font to render the non-English text. The Unicode project only specifies numbered codepoints for its characters. The actual visual representation of the characters is left to the font engine and fonts installed in the output device. For example, in the Malayalam letter ‘Shree’, there are four Unicode characters — ෂ (0D36), ് (0D4D), ൃ (0D30), and ൄ (0D40). A font that supports Malayalam decides how to combine them as ്. You will not find the finally rendered ligature in Unicode. If you move the cursor across the letter, you will be able to split it into two ligatures ് and ൃ. Even these will not be found in Unicode. How they are formed is left to the font maker.

You will have to view a ligature in a hex editor to identify the individual codepoints. Then, you can search the codepoints in the Unicode list. That will tell you what language the character belongs to. You can then install a font that supports that language.

The full character list is published on the Unicode website at <http://ftp.unicode.org/Public/UNIDATA/UnicodeData.txt>. This file tips the scale at nearly 2MB. It is in the form of a comma-separated file (CSV). With some JavaScript (<http://www.vsubhash.in/refs/unicode.html>), you can render it as a table. (The following code will tax the JavaScript and HTML engines of a browser. Do not run it on a system that is light on resources.)

```
<!doctype html>
<html>
  <head>
    <meta http-equiv="Content-Type" content="text/html;
    charset=UTF-8" />
    <link href="universal.css" rel="stylesheet">
  <script>

    // Make a copy of the Unicode data text file in the
```